Clase 11 · Ciclos y retardos en una máquina de estado PIO

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · miércoles 14 oct 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Clase breve (el profesor andaba con mucha carga). Repite paso a paso el cálculo de la Clase 10.

1. PIO frente al ensamblador de la CPU y la FPU

- Con las **máquinas de estado PIO** se manejan los pines directamente, sin tantas instrucciones complejas como en el ensamblador de la CPU.
- La **FPU** en ensamblador sirve para programas matemáticos (proyecto de Taylor, Clase 10), pero es incómoda e ineficiente para la E/S de pines.

Regla: FPU para cálculo; PIO para pines y tiempos estrictos.

2. Instrucciones, etiquetas y retardos

- Las **etiquetas** (`label("delay")`) y los nombres de subrutina no se ejecutan (el profesor dijo que se pueden contar como la "instrucción número 10").
- Un retardo se escribe dentro de la misma instrucción, entre corchetes (tope de 31–32 ciclos por el campo de bits).

Instrucción	Ciclos
`nop() [29]`	1 + 29 = **30**
`jmp(x_dec, "delay")`	**1** (decrementa X y salta si X no era 0)
`set(x, 31) [6]`	1 + 6 = 7
`set(pins, 1)`	1

Error común: olvidar sumar el ciclo de la instrucción al número entre corchetes.

3. Cálculo exacto de ciclos

Fase encendida

Preparación: `set(pins, 1)` (1) + `set(x, 31) [6]` (7) = **8 ciclos**. X empieza en 31 y el bucle corre hasta pasar por 0: **X + 1 = 32 vueltas**. Cada vuelta: `nop() [29]` (30) + `jmp` (1) = **31 ciclos**. Bucle: 32 × 31 = 992. **Total: 8 + 992 = 1000 ciclos.**

- **Fase apagada:** espejo con `set(pins, 0)` y la etiqueta `delay_low`: otros 1000 ciclos.
- **Periodo completo:** 1000 + 1000 = **2000 ciclos**.

La rutina es un divisor exacto de la frecuencia asignada a la máquina. Error común: contar 31 vueltas en lugar de 32.

4. La máquina de estado en MicroPython

```
SM = rp2.StateMachine(1, LED, freq=2000, set_base=Pin(25))
```

Parámetro	Significado
`1`	Número de máquina de estado
`LED`	La rutina decorada con `@rp2.asm_pio`
`freq=2000`	Reloj local de la máquina: 2000 Hz
`set_base=Pin(25)`	Pin de salida

Frecuencia de destello

Periodo = ciclos de la rutina ÷ freq = 2000 ÷ 2000 = **1 s (1 Hz)**: 0.5 s encendido y 0.5 s apagado. No confundir `freq` (reloj de la máquina) con la frecuencia del parpadeo.

- Reloj del sistema: **125 MHz (Pico 1)** y **150 MHz (Pico 2)**; el **bootloader** ajusta los divisores para dar los 2000 Hz pedidos. Según el profesor, con el reloj del sistema a 300 MHz destellaría cada medio segundo.

5. Control desde Python

```
while True:
    SM.active(1)     # activa la máquina: 4 s → 4 destellos
    time.sleep(4)
    SM.active(0)     # la desactiva: LED apagado 4 s
    time.sleep(4)
```

Mientras la máquina trabaja, la CPU queda libre. Con varias máquinas, cada una necesita su propia variable (SM1, SM2…).

6. PIO frente a la arquitectura Harvard de la CPU

Propiedad	Detalle

Salto en 1 ciclo	En la CPU (Harvard) un salto cuesta ≈ 3 ciclos mínimo; en la PIO, `JMP` siempre dura 1
Registros X y Y	Dos registros de 32 bits por máquina; se decrementan (`x_dec`, `y_dec`) o se usan como condición
Instrucciones efectivas	De las 9–10, 2 son para pila/FIFO: quedan **7** para pines y periféricos
Paralelismo y DMA	Corren en paralelo entre sí y con los núcleos; cuentan eventos externos y se comunican por DMA

Por eso la PIO emula UART, SPI o video a varios MHz sin perder sincronía. Próxima clase: contador para medir frecuencia con máquinas de estado.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

> **Cálculo PIO (examen)**
>
> Preparación 8 ciclos + 32 vueltas × 31 ciclos = **1000 por fase**; 2000 por periodo. `set(x, 31)` da 32 vueltas; `nop() [29]` = 30 ciclos; `jmp` = 1.

Frecuencia. Periodo = ciclos ÷ freq → 2000 ÷ 2000 Hz = 1 s (1 Hz). Pico 1: 125 MHz; Pico 2: 150 MHz; el bootloader ajusta los divisores.

PIO vs. CPU	Para recordar
Saltos	PIO 1 ciclo; CPU Harvard ≈ 3
Registros	X y Y de 32 bits
Instrucciones	7 efectivas (2 de pila/FIFO)
Ejecución	Paralela a la CPU; DMA
`SM.active(1/0)`	Encender/apagar la máquina desde Python